UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

JAVASCRIPT

JavaScript is the scripting language of the Web.

JavaScript is used in millions of Web pages to add functionality, validate forms, detect browsers, and much more.

Introduction to JavaScript

JavaScript is a versatile, dynamically typed programming language that brings life to web pages by making them interactive.

It is used for building interactive web applications, supports both client-side and server-side development, and integrates seamlessly with HTML, CSS, and a rich standard library.

What we can do with JavaScript?

- Designing Dynamic Web Pages
- Client-Side validation
- Change the HTML content

UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

- Change the Attributes
- Change the CSS
- Hide and Display the content

CHANGE THE HTML CONTENT

*Untitled - Notepad

File Edit Format View Help

```
<html>
<body>
<h1>CHANGE THE HTML CONTENT</h1>
<h2 id="hcontent">Hello Guest!<h2>
<h3>Welcome to Java Script</h3>
<button type="button" onclick='document.getElementById("hcontent").innerHTML="Hello Sundeep!"'>Sundeep</button>
<button type="button" onclick='document.getElementById("hcontent").innerHTML="Hello Saradhi!"'>Saradhi</button>
</body>
</html>
```

UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

CHANGE IN ATTRIBUTE

```
<h1>CHANGE IN ATTRIBUTES</h1>
<img id="image" src="red.jpg" width="100" height="100"><br><br>
<button type="button" onclick='document.getElementById("image").src="green.png" '>Green</button>
<button type="button" onclick='document.getElementById("image").src="blue.png" '>Blue</button>
<button type="button" onclick='document.getElementById("image").src="red.jpg" '>Red</button>
```

CHANGE IN CSS

```
<h1>CHANGE IN CSS</h1>
<h2 id="text">Introduction to Java Script</h2>
<button type="button" onclick='document.getElementById("text").style.color="green" '>Green</button>
<button type="button" onclick='document.getElementById("text").style.color="blue" '>Blue</button>
```

HIDE AND DISPLAY

```
<h1>HIDE AND DISPLAY CONTENT</h1>
<h2 id="hidedisplay">This will demonstrate the hiding and displaying the content</h2>
<button type="button" onclick='document.getElementById("hidedisplay").style.display="none" '>Hide</button>
<button type="button" onclick='document.getElementById("hidedisplay").style.display="block" '>Display</button>
</body>
</html>
```

UNIT 2

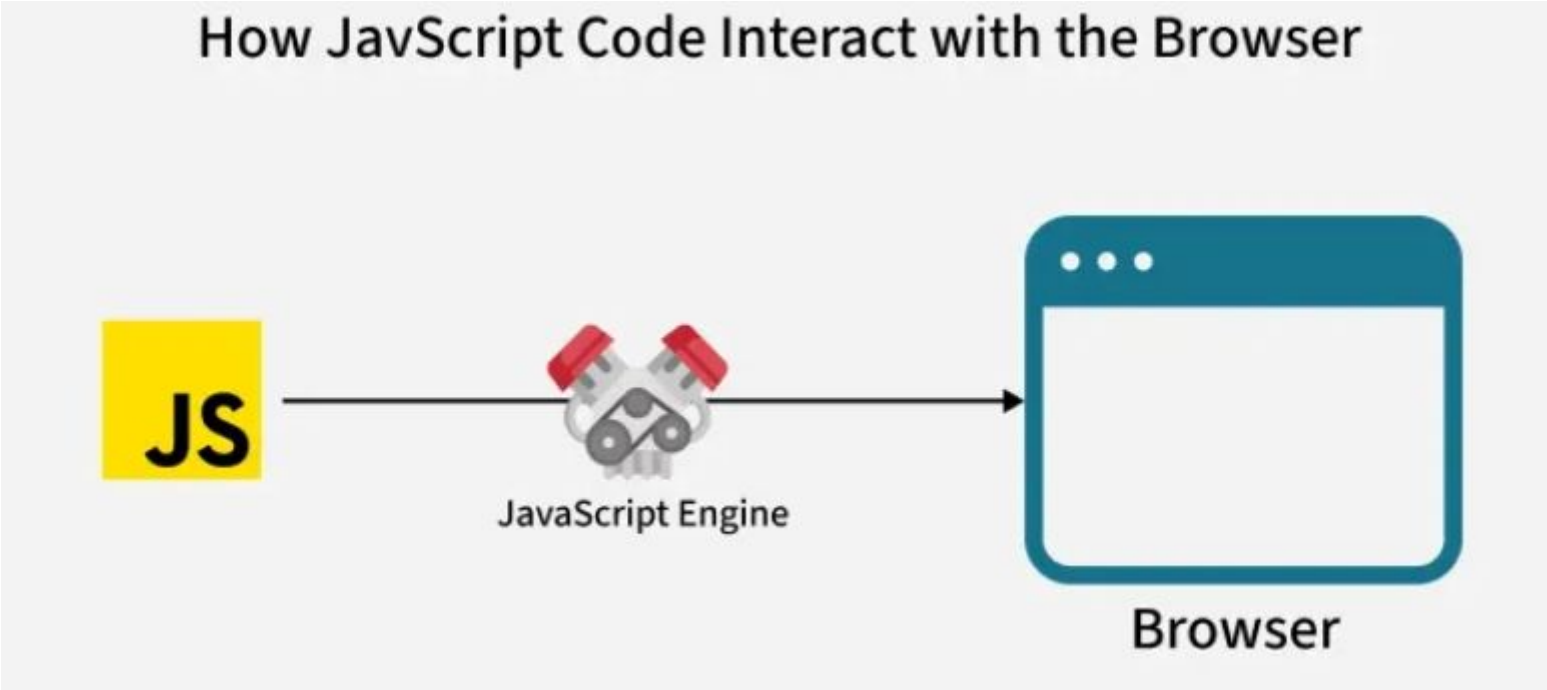
CLIENT SIDE PROGRAMMING – JAVASCRIPT

JavaScript Code Execution

*Javascript was initially designed to run within a web browser that has a Javascript engine.

*This engine is built into most major browsers. These are like V8 on Google Chrome, Mozilla Firefox has SpiderMonkey, and so on.

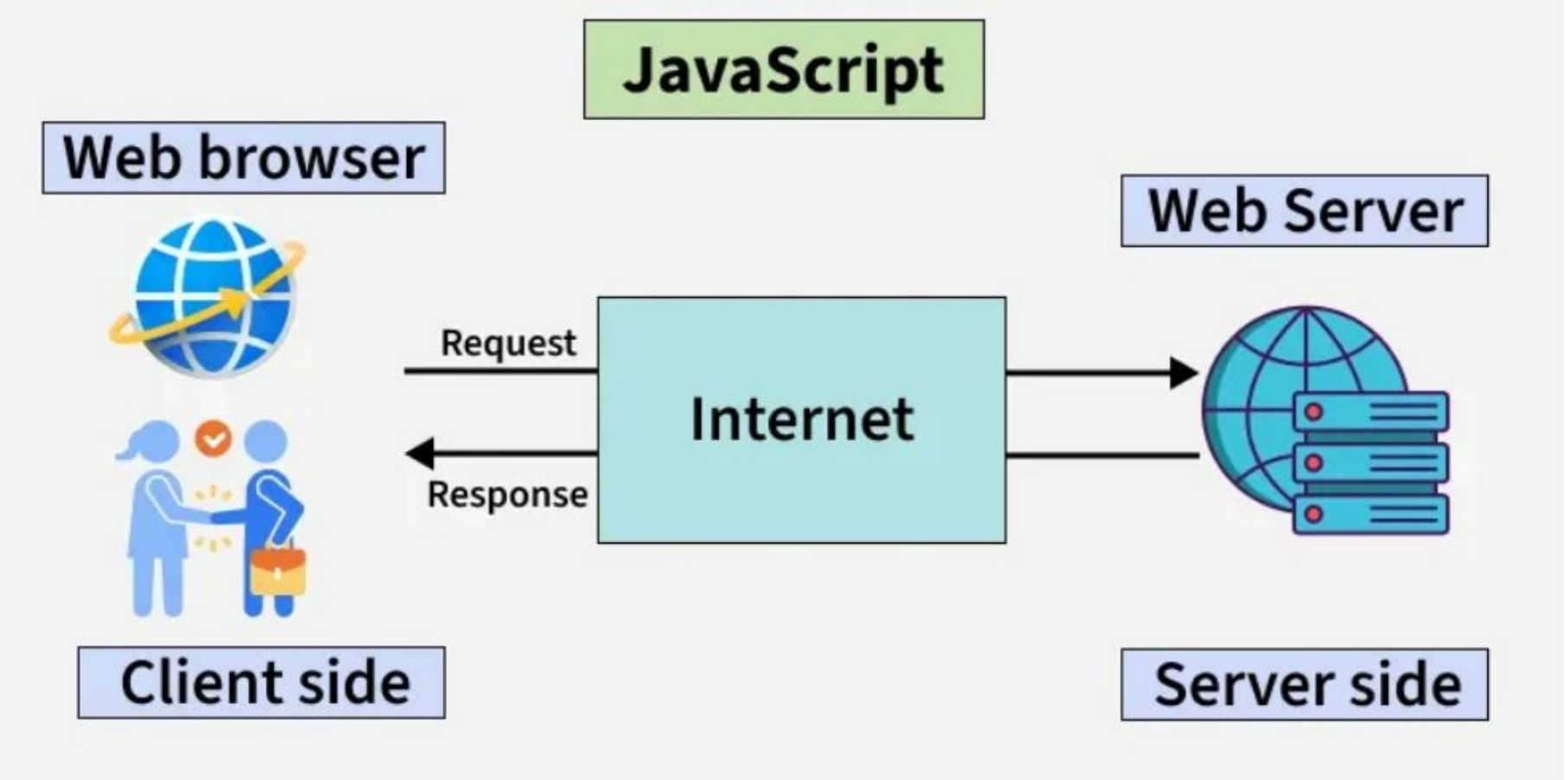
*They all are used for the same purpose because the browser cannot directly understand Javascript code.



UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

Client Side and Server Side nature of JavaScript



UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT



"Hello, World!" Program in Browser Console

```
<html>
<head></head>
<body>
    <h1>Check the console for the message!</h1>
    <script>
        // This is our first JavaScript program
        console.log("Hello, World!");
    </script>
</body>
</html>
```

console.log() prints messages to the browser's developer console. Open the browser console to see the "Hello, World!" message.

UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

How to Add JavaScript in HTML Document?

Inline JavaScript

write JavaScript code directly inside the HTML element using the onclick, onmouseover, or other event handler attributes.

```
<html>
<head></head>
<body>
    <h2>
        Adding JavaScript in HTML Document
    </h2>
    <button onclick="alert('Button Clicked!')">
        Click Here
    </button>
</body>
</html>
```



UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

Internal JavaScript (Within <script> Tag)

write JavaScript code inside the <script> tag within the HTML file. This is known as internal JavaScript and is commonly placed inside the <head> or <body> section of the HTML document.

```
<html>
<head>
    <script>
        function myFun() {
            document.getElementById("demo")
                .innerHTML = "Content changed!";
        }
    </script>
</head>
<body>
    <h2>
        Add JavaScript Code
        inside Head Section
    </h2>
    <h3 id="demo" style="color:green;">
        GeeksforGeeks
    </h3>
    <button type="button" onclick="myFun()">
        Click Here
    </button>
</body>
</html>
```

```
<html>
<head></head>
<body>
    <h2>
        Add JavaScript Code
        inside Body Section
    </h2>
    <h3 id="demo" style="color:green;">
        GeeksforGeeks
    </h3>
    <button type="button" onclick="myFun()">
        Click Here
    </button>
    <script>
        function myFun() {
            document.getElementById("demo")
                .innerHTML = "Content changed!";
        }
    </script>
</body>
</html>
```

External JavaScript (Using External File)

For larger projects or when reusing scripts across multiple HTML files, you can place your JavaScript code in an external .js file. This file is then linked to your HTML document using the src attribute within a <script> tag.

```html
<html>
<head>
    <script src="script.js"></script>
</head>
<body>
    <h2>
        External JavaScript
    </h2>
    <h3 id="demo" style="color:green;">
        GeeksforGeeks
    </h3>
    <button type="button" onclick="myFun()">
        Click Here
    </button>
</body>
</html>
```

```javascript
/* Filename: script.js*/

function myFun () {
    document.getElementById('demo')
        .innerHTML = 'Content Changed'
}
```

UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

JavaScript Output

1. JavaScript innerHTML Property

The innerHTML property is used with the HTML element. JavaScript can be used to select an element using the document.getElementById(id) method, and then use the innerHTML property to display the output on the specified element (selected element).

Syntax:

```
document.getElementById("id").innerHTML;
```

2. JavaScript console.log() Method

The console.log() method is used for logging messages to the console. It is a debugging tool available in most web browsers. It is used during development to output information about the state of the program.

```
console.log();
```

3. JavaScript document.write() Method

To use the document.write(), simply call this method and provide the content you want to be written to the document. This method is often used for directly adding content to the HTML document while it is being parsed.

Syntax:

```
document.write();
```

4. JavaScript window.alert() Method

The window.alert() method is used to display an alert box with a specified output (or message) and an OK button.

Syntax

```
window.alert();
```

5. JavaScript window.print() Method

The window.print() method is used to open the browser's print dialog, allowing the user to print the current page. JavaScript does not contain any default print object or methods.

Syntax

```
window.print();
```

UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

Variables

Variables are "containers" for storing information.

JavaScript variables are used to hold values or expressions.

A variable can have a short name, like x, or a more descriptive name, like carname.

Rules for JavaScript variable names:

Variable names must begin with a letter or the underscore character

Variable names should not have any other special symbols other than an underscore.

Variable names should not match keywords, such as 'break' and 'continue'.

Variable names do not contain hyphens (-).

Variable names are case sensitive (y and Y are two different variables)

Declaration

Keyword: var, let, const

Keyword variablename;

let a;

Intialization

Keyword variablename = value;

let a = 10;

UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

1. var Keyword

The var keyword is used to declare a variable. It has a function-scoped or globally-scoped behaviour.

```
var n = 5;
console.log(n);
var n = 20; // reassigning is allowed
console.log(n);
```

Output

```
5
20
```

2. let Keyword

The let keyword is introduced in ES6, has block scope and cannot be re-declared in the same scope.

```
let  n= 10;
n = 20; // Value can be updated
// let n = 15; //can not redeclare
console.log(n)
```

output

```
20
```

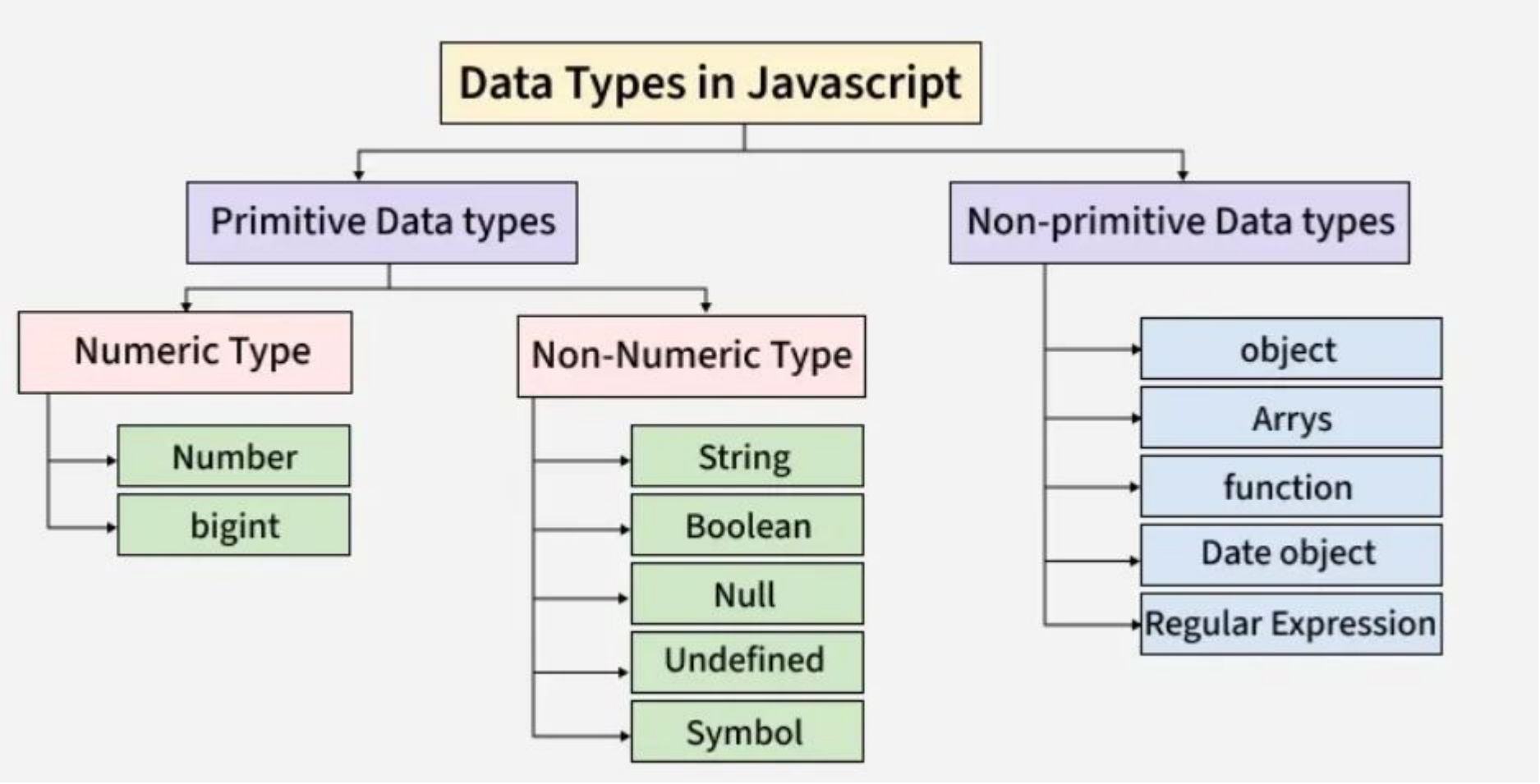
3. const Keyword

The const keyword declares variables that cannot be reassigned. It's block-scoped as well.

```
const n = 100;
// n = 200; This will throw an error
console.log(n)
```

UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT



UNIT 2

CLIENT SIDE PROGRAMMING – JAVASCRIPT

Primitive Datatypes

Primitive datatypes represent single values and are immutable.

1. Number: Represents numeric values (integers and decimals).

```
let n = 42;
let pi = 3.14;
```

2. String: Represents text enclosed in single or double quotes.

```
let s = "Hello, World!";
```

3. Boolean: Represents a logical value (true or false).

```
let bool= true;
```

4. Undefined: A variable that has been declared but not assigned a value.

```
let notAssigned;
console.log(notAssigned);
```

5. Null: Represents an intentional absence of any value.

```
let empty = null;
```

6. Symbol: Represents unique and immutable values, often used as object keys.

```
let sym = Symbol('unique');
```

7. BigInt: Represents integers larger than Number.MAX_SAFE_INTEGER.

```
let bigNumber = 1234567890123456789012345678890n;
```

Non-Primitive Datatypes

1. Object: Represents key-value pairs.

```
let obj = {
    name: "Amit",
    age: 25
};
```

2. Array: Represents an ordered list of values.

```
let a = ["red", "green", "blue"];
```

3. Function: Represents reusable blocks of code.

```
function fun() {
    console.log("GeeksforGeeks");
}
```

UNIT 2

TYPE CONVERSION

Type Conversion / Type Casting

Converting one datatype to another datatype

1. Number → String → Boolean

```
let a = 23;
console.log(a);
Console.log(typeof a);
let b = String(a);        → Built in function
Console.log(b);
Console.log(typeof b);
```

23
number
23
String

2. ~~Boolean~~ → ~~Boolean~~ String

```
let a = false;
Console.log(a);
Console.log(typeof a);
let b = String(a);
Console.log(a);
Console.log(typeof b);
```

3. String → Number
(alphanumeric)

```
let a = "123";
Console.log(a);
Console.log(typeof a);
let b = Number(a);
Console.log(b);
Console.log(typeof b);
```

Stack
a = "123"
Type: String

Activity

let a = "123 sandhiya"
What will be the output?

Answer: 123 sandhiya
String
NAN
Number

The More option is there for converting string into Number

parseInt() checking from left to right

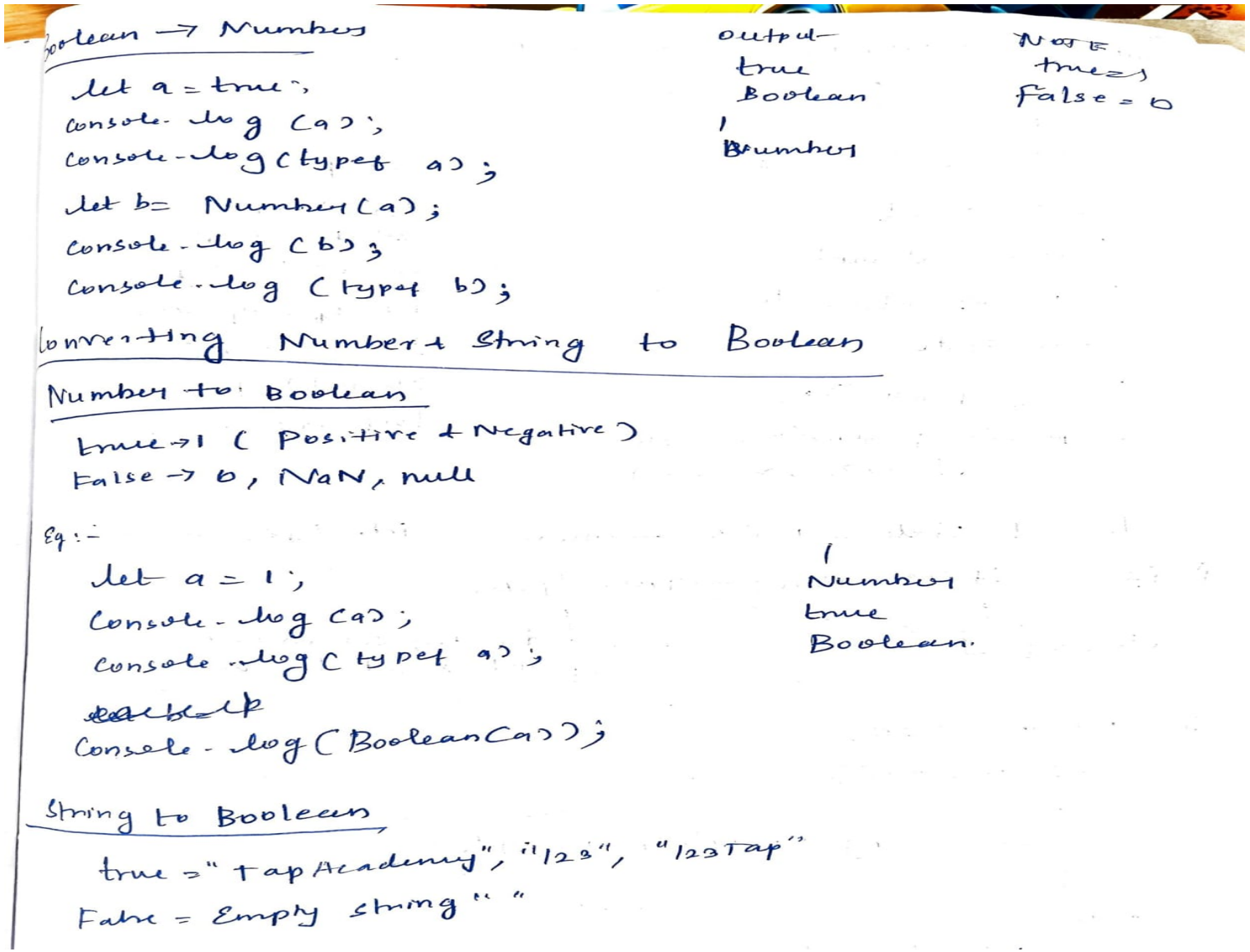
```
let a = "123Tap"
Console.log(a);
Console.log(typeof a);
let b = parseInt(a);
Console.log(b);
Console.log(typeof b);
```

123Tap
string
123
Number

Activity :- "Tap123" ⇒ NaN
"123" ⇒ ~~parseInt~~ Number
"123Tap" ⇒ 123 Number

UNIT 2

TYPE CONVERSION



Boolean → Number

```
let a = true;
Console.log(a);
Console.log(typeof a);
let b= Number(a);
Console.log(b);
Console.log(typeof b);
```

Output:
true
Boolean
1
Number

Note:
true → 1
false = 0

Converting Number & String to Boolean

Number to Boolean

true → 1 (Positive & Negative)
False → 0, NaN, null

Eg:-

```
let a = 1;
Console.log(a);
Console.log(typeof a);
Console.log(Boolean(a));
```

1
Number
true
Boolean.

String to Boolean

true = "Tap Academy", "123", "123Tap"
False = Empty string " "

JavaScript Loops

Loops in JavaScript are used to reduce repetitive tasks by repeatedly executing a block of code as long as a specified condition is true. This makes code more concise and efficient.

Example:
we want to print 'Hello World' five times

1. JavaScript for Loop

The for loop repeats a block of code a specific number of times. It contains initialization, condition, and increment/decrement in one line.

Syntax

```
for (initialization; condition; increment/decrement) {
    // Code to execute
}
```

Example: ouput

```
for (let i = 1; i <= 3; i++) {
    console.log("Count:", i);
}
```

```
Count: 1
Count: 2
Count: 3
```

2. JavaScript while Loop

Syntax

```
while (condition) {
    // Code to execute
}
```

```
let i = 0;
while (i < 3) {
    console.log("Number:", i);
    i++;
}
```

```
Number: 0
Number: 1
Number: 2
```

3. JavaScript do-while Loop

The do-while loop is similar to while loop except it executes the code block at least once before checking the condition.

```
do {
    // Code to execute
} while (condition);
```

```
let i = 0;
do {
    console.log("Iteration:", i);
    i++;
} while (i < 3);
```

```
Iteration: 0
Iteration: 1
Iteration: 2
```

4. JavaScript for-in Loop

The for...in loop is used to iterate over the properties of an object. It only iterate over keys of an object which have their enumerable property set to “true”.

Syntax

```
for (let key in object) {
    // Code to execute
}
```

output

```
const obj = { name: "Ashish", age: 25 };
for (let key in obj) {
    console.log(key, ":", obj[key]);
}
```

5. JavaScript for-of Loop

The for...of loop is used to iterate over iterable objects like arrays, strings, or sets. It directly iterate the value and has more concise syntax than for loop.

```
for (let value of iterable) {
    // Code to execute
}
```

```
let a = [1, 2, 3, 4, 5];
for (let val of a) {
    console.log(val);
}
```

```
1
2
3
4
5
```

Functions in JavaScript

Functions in JavaScript are reusable blocks of code designed to perform specific tasks. They allow you to organize, reuse, and modularize code. It can take inputs, perform actions, and return outputs.

Function Syntax and Working

- Begin with the keyword **function** Keyword
- A user-defined function name
- A list of parameters enclosed within parentheses and separated by commas
- A list of statements composing the body of the function enclosed within curly braces {}

Syntax:

```
function name(arguments)
{
   javascript statements
}
```

Function Calling:

Function Calling at a later point in the script, simply type the function's name.

```
function welcome() {
    console.log("welcome to GfG");
}

// Function calling
welcome();
```

Function Arguments:

A function can contain one or more arguments that are sent by the calling code and can be utilized within the function.

JavaScript is a dynamically typed programming language, a **Function Arguments** can have any data type as a value.

```
function welcome(name) {
    console.log("Hey " + "" + name + " " + "welcome to GfG");
}

// Passing arguments
welcome("Rohan");
```

```
Hey Rohan welcome to GfG
```

Return Value:

A return statement is an optional part of a JavaScript function.

If you wish to return a value from a function, you must do this.

Return Value should be the final statement of a function.

```
function welcome() {

    // Return statement
    return "Welcome to GfG";
}

console.log(welcome());
```

UNIT 2

Types of functions

① No input & No output

Eg 1: Add two numbers

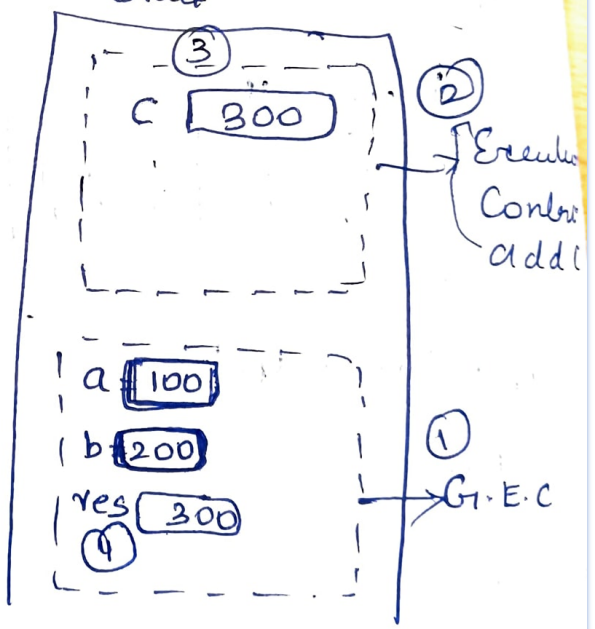
```
let a = 100;
let b = 200;

function add()
  {
    let c;
    c = a+b;
    console.log(c);
  }

add(); // calling a function
```

② No input and returning the output

```
let a=100;
let b= 200;
function add()
{
  let c;
  c = a+b;
  return c;
}
let result = add();
console.log(result);
```



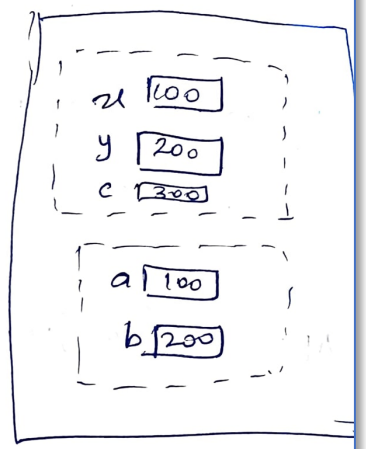
Return then we sho
collect

UNIT 2

③ Input and Not returning the output

100 200
How to collect of these two number so need variable e.g (x, y)

```
function add (x, y)
{
  let c ;
  c = x + y ;
  console.log (c);
}
```



Start.

One way

```
add (100, 200);
```

(or)

```
let a = 100;
let b = 200;
add (a, b)
```

① Accept input and returning output

```
function add (x, y)
{
  let c ;
  c = x + y ;
  return c ;
}
let a = 100;
let b = 200;
let res = add (a, b);
console.log (res);
```